

Deep Learning Homework 2

Afonso Luís Vieira Gouveia Catarina Sofia da Silva Gonçalves

ist1102563

ist1103989

André Antunes Amaro

ist1106419

afonso.v.gouveia@tecnico.ulisboa.pt

catarina.s.goncalves@tecnico.ulisboa.pt

andre.amaro@tecnico.ulisboa.pt

Abstract

We address two deep learning challenges: medical image classification and genomic sequence regression. In the first task, we implement Convolutional Neural Networks to classify blood cell images from the BloodMNIST dataset. Our experiments include adding an explicit Softmax layer before Cross-Entropy loss and integrating Maxpooling layers. In the second task, we develop deep regression models to predict the binding affinity of the RBFOX1 protein from RNA sequences. We compare a 1D-CNN architecture, against a Bidirectional LSTM (Bi-LSTM). We investigate the impact of attention mechanisms on model performance. Finally, we explore a scalable dual branch architecture to generalize binding predictions across multiple proteins.

Afonso Gouveia solved questions 2.1 and 2.2. Catarina Gonçalves solved questions 2.1 and 2.2. André Amaro solved questions 1.1, 1.2 and 2.3.

1 Question 1

1.1

Both models achieved a peak validation accuracy of approximately 0.94, with training loss converging rapidly from 0.7 to near 0 within the first 25 epochs. Interestingly, both configurations exhibited a similar instability event during the later stages of training (around epoch 110 for the model without Softmax and epoch 145 for the model with Softmax) characterized by a sharp spike in loss and a temporary drop in accuracy, from which both fully recovered.

The configuration without a Softmax layer is the standard approach because `nn.CrossEntropyLoss` applies `LogSoftmax` internally. In contrast, adding an explicit Softmax layer results in a double application of the function, effectively compressing outputs into the 0-1 range

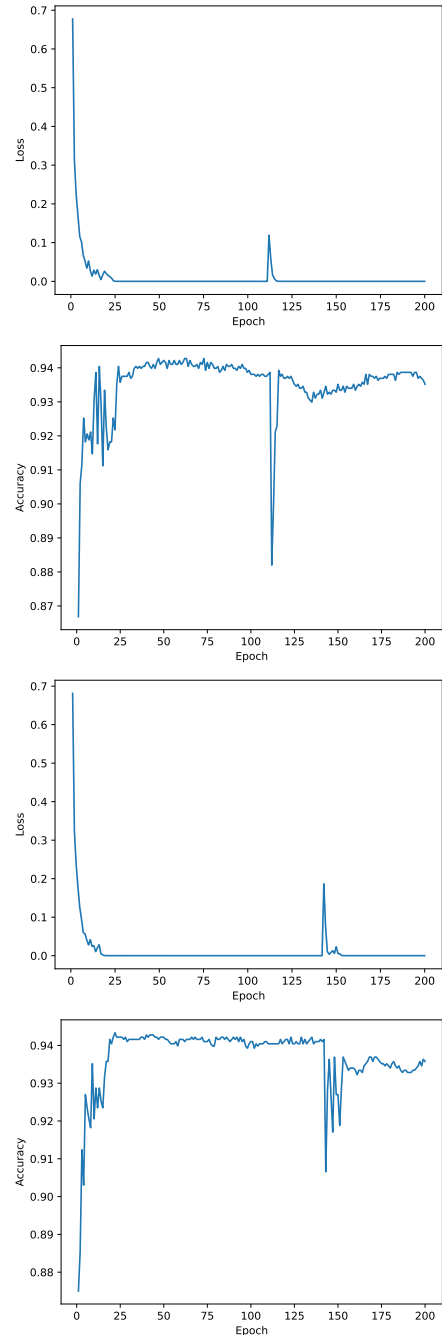


Figure 1: Training loss and validation accuracy per epoch for the models with and without a Softmax layer at output.

before the loss calculation. While the optimizer successfully handled this redundancy in this experiment, this approach is less numerically stable and carries a higher risk of vanishing gradients.

Despite these internal differences, the final classification performance was identical because prediction relies on the argmax function. Since Softmax is monotonic, it preserves the relative order of the class scores. Therefore, the index of the highest value remains the same whether applied to raw logits or Softmax probabilities. The test accuracy for the best validation model without Softmax was 0.9298, and with Softmax was 0.9316.

Regarding the model’s suitability for the task, it seems to have sufficient capacity to capture the relevant morphological features of the blood cells without significant overfitting. While the observed intermittent loss spikes suggest that the training dynamics could benefit from minor hyperparameter tuning (such as learning rate scheduling), the model’s ability to recover and maintain high performance confirms it is a robust and efficient baseline for this classification problem.

1.2

The introduction of `nn.MaxPool2d(2)` layers resulted in a tangible improvement. The configurations with and without Softmax achieved a peak validation accuracy of approximately 0.95, slightly surpassing the 0.94 peak observed in the architecture without pooling. The test accuracy for the best validation model without Softmax was 0.9468, and with Softmax was 0.9474.

This suggests that the translation invariance and feature abstraction provided by max-pooling allowed the model to generalize better to unseen data and capture essential morphological features while becoming less sensitive to their exact spatial location. Similar to the previous experiments, both setups experienced transient instability late in training, characterized by a sudden loss spike, but successfully recovered their high accuracy immediately. The total training time on a Kaggle P100 GPU dropped from approximately 15 minutes for the original architecture to roughly 10 minutes for the pooled models, a reduction of about 33%. This is driven by the drastic reduction in spatial dimensions after each block ($28 \rightarrow 14 \rightarrow 7 \rightarrow 3$), which shrank the feature map volume entering the first linear layer from 100,352 to just 1,152 features. Consequently, the number of parameters in the first fully connected layer fell from approximately 25.7

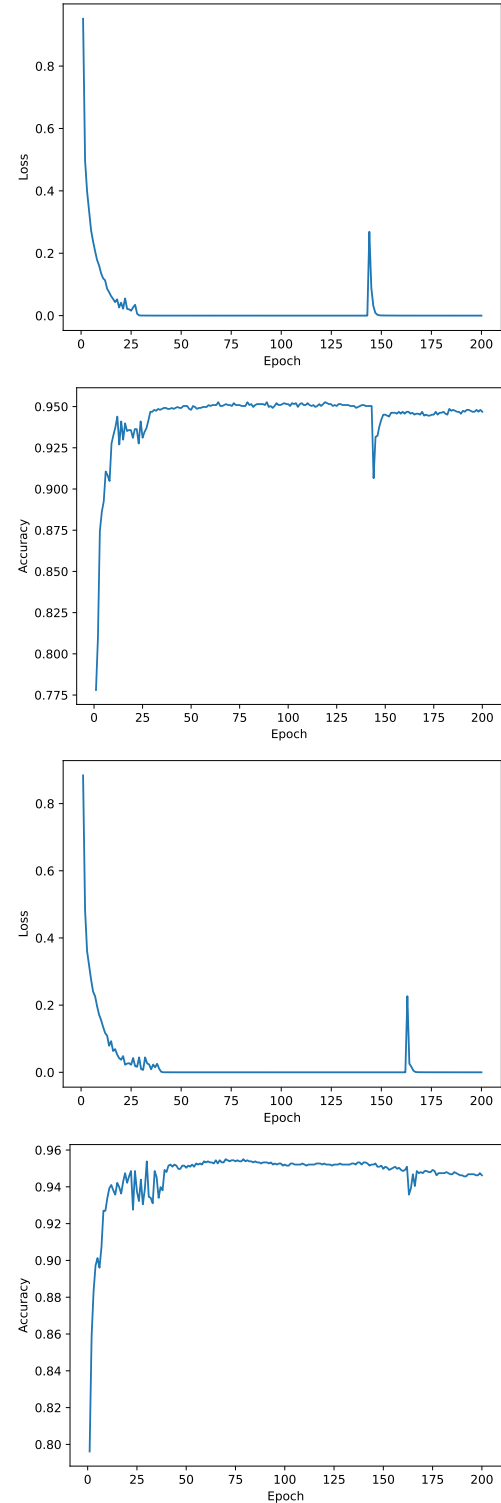


Figure 2: Training loss and validation accuracy per epoch for the models with Maxpooling and with and without a Softmax layer at output.

million to roughly 0.3 million, significantly lowering the computational cost and memory usage per epoch.

2 Question 2

2.1

The models we experimented with were a one dimensional Convolutional Neural Network (1D-CNN) inspired by the DeepBind architecture (Alipanahi et al., 2015; Zeng et al., 2016), and a bidirectional LSTM (Bi-LSTM).

2.1.1 1D-CNN

The choice of this model was biologically motivated. The convolutional layers mimic the physical scanning process of the protein, where learned filters act as motif detectors analogous to Position Weight Matrices (PWMs). Furthermore, since RBPs bind if the motif is present anywhere in the sequence, the model's global max pooling layer replicates this behavior by outputting the strongest signal found, regardless of its position.

To ensure that the model learns exclusively from biological signals and not from padding artifacts, a custom class `MaskedGlobalMaxPool1d` was implemented. This layer identifies convolution windows that overlap with padding tokens by applying a max-pooling operation on the inverted binary mask. Outputs corresponding to these "contaminated" windows are suppressed (set to -1) prior to global pooling, ensuring that the network never updates its weights based on artificial padding (as the ReLU activation function is used after the convolution phase).

The hyperparameter selection strategy relied on automated optimization using Optuna (Akiba et al., 2019). The kernel size was searched as an integer within the range [5, 9], chosen to cover the 6-nucleotide RBFOX1 motif while allowing for wider context windows. To enable the detection of multiple strong binding sites per sequence, the pooling parameter k was optimized in the range [1, 3]. The number of convolutional filters and hidden units were searched within the standard range [32, 128]. Finally, the learning rate was searched in a log-uniform distribution between 10^{-4} and 10^{-2} and the dropout probability between [0, 0.5] were optimized to ensure stable convergence and regularization.

2.1.2 Bi-LSTM

While the 1D-CNN focuses on detecting local sequence patterns (motifs), the Bi-LSTM was chosen to model the whole RNA sequence. RNA Binding Proteins act like a "key" seeking a specific "lock". This "lock" is defined not only by specific nucleotide patterns (e.g, *UGCAUG* motif for RBFOX1) but also by the structural shapes of the RNA molecule. Because of this it, makes sense to consider a model that is able to consider the full context rather than just isolated parts like LSTM. The bidirectional architecture allows the network to process the sequence in both directions, enabling it to learn the structural context that creates the correct "shape" for binding, rather than just identifying the presence of the motif characters.

To generate a single binding score from the entire sequence, we used a max pooling operation. Since the sequences have variable lengths and are padded with zeros, a standard max pooling layer might incorrectly select the padding values (0) if the model predicts negative scores for the real data. To prevent this, we explicitly masked the padding positions by setting their values to a large negative number before pooling. This forces the model to ignore the padding and select the strongest signal exclusively from the valid RNA nucleotides.

The hyperparameter selection strategy relied on automated optimization using Optuna. The hidden state dimension was searched as an integer within the range [32, 128], chosen to provide sufficient memory capacity to track dependencies across the sequence without introducing excessive parameters. To balance model complexity against the risk of overfitting, the number of recurrent layers was optimized in the range [1, 2]. Finally, the learning rate was searched in a log-uniform distribution between 10^{-4} and 10^{-2} and the dropout probability between [0, 0.5] were optimized to ensure stable convergence and regularization.

2.1.3 Expectations

We expected that the Bi-LSTM will outperform the 1D-CNN. While the 1D-CNN detects the presence of the binding motif, it treats the RNA as a flat sequence. However, biological binding is governed by structural accessibility, so a motif cannot be found if it is buried within a folded structure. The Bi-LSTM processes the entire sequence context globally, allowing it to implicitly infer these structural constraints and predict not just if the motif exists, but if it is accessible to the protein.

2.1.4 Results

The automated hyperparameter optimization using Optuna identified distinct optimal configurations for each model. The selected hyperparameters are summarized in the following table.

Parameter	1D-CNN	Bi-LSTM
Kernel Size	6	-
Filters	126	-
Pooling (k)	2	-
Layers	-	2
Hidden Dimension	42	83
Dropout Rate	0.12	0.28
Learning Rate	1.2×10^{-3}	1.3×10^{-3}

Table 1: Optimal hyperparameters selected by Optuna.

Note that the 1D-CNN selected a kernel size of 6, which exactly matches the length of the RBFOX1 consensus motif, and a pooling parameter of $k = 2$, suggesting the model benefits from detecting multiple binding sites per sequence. For the Bi-LSTM, the optimization preferred a deeper architecture with 2 stacked layers and a higher dropout rate compared to the CNN, ensuring adequate regularization for the increased parameter space.

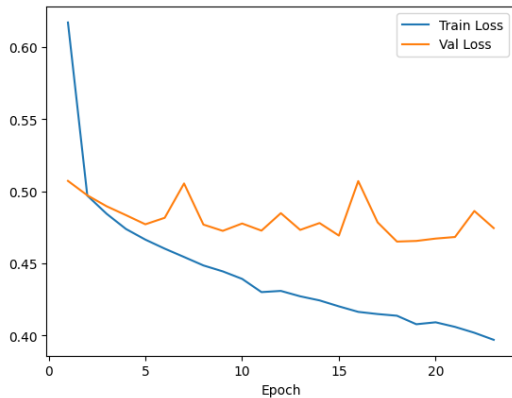


Figure 3: 1D-CNN training and validation loss curves.

The validation performance aligned with our initial expectations. The Bi-LSTM achieved a Spearman rank correlation of 0.662, significantly outperforming the 1D-CNN, which reached 0.566. As illustrated by the convergence plots in Figure 3 and Figure 4, both models stabilized, but the Bi-LSTM reached a lower validation loss. This superiority confirms that modeling the global sequence, rather than just local motif detection, is crucial for predicting RNA.

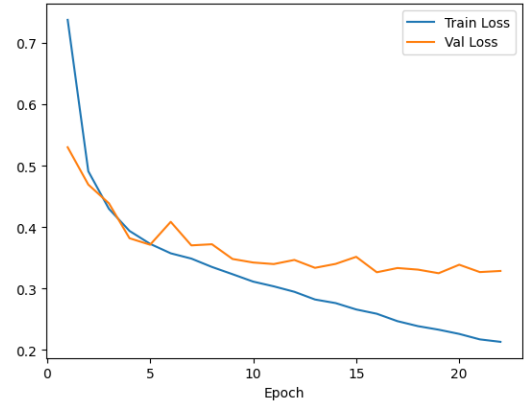


Figure 4: Bi-LSTM training and validation loss curves.

2.2

2.2.1

The Bi-LSTM model was chosen to include attention, as it has proven more capable to correctly predict the affinity on this regression problem. A single attention head followed by attention pooling on each contextualized representation for the i -th nucleotide were chosen for this task: in spite of this (symmetry aware) RNN architecture variant being able to memorize local patterns, providing global attention to the RNA sequence allows a more judgmental analysis of a nucleotide with respect to the global characteristics of the RNA sequence. To avoid using a model with too much capability (prone to overfitting), the attention pool maintained a more conservative architecture with respect to interpretability and overfitting. We expect that this new variant could prove a somewhat improved results.

2.2.2 Results

The best hyperparameter configuration of this model is shown on the table below.

Parameter	Bi-LSTM with attention
Layers	2
Hidden Dimension	97
Dropout Rate	0.28
Learning Rate	9.7×10^{-4}

Table 2: Optimal hyperparameters selected by Optuna.

The model obtained a Spearman correlation of 0.666, representing only a negligible improvement over the baseline score of the Bi-LSTM model 2.1.2 previously implemented. We hypothesize that RNA sequences are of insufficient length (up to 41 nucleotides) for the model to effectively forget any relevant information throughout each layer. As a

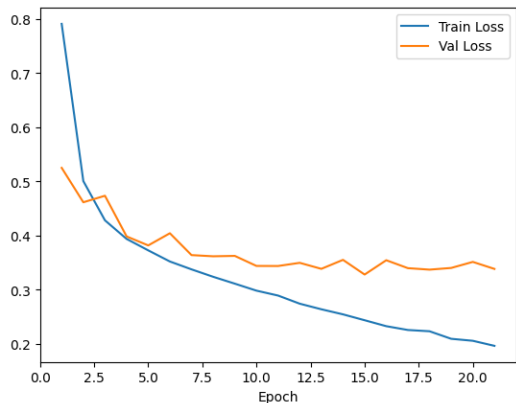


Figure 5: Bi-LSTM (with attention) training and validation loss curves.

result the attention layers are not adding any relevant information. This is further corroborated by the similarities between the optimized hyperparameters (Tables 1 and 2) and validation loss curves (Figures 4 and 5) between both Bi-LSTM variant models.

2.3

2.3.1 Proposed modifications

The data input needs to include the protein representation along the RNA sequence. A way to represent this could be through a change from a single tensor to a tuple of two tensors – the RNA and an integer index for the protein.

Regarding the label modification, since different experiments for different proteins might use distinct scales for affinity, Z-score normalization, clipping and log-transforming could prove to be even more important, so the model doesn't bias towards proteins with naturally higher raw affinity scores. Thus, as the provided pipeline already supports this setup, there would be no need for change. Additionally, we must retain the mask 2 provided by the loader to handle NaN values across all proteins.

For the neural network architecture, we propose a multi-modal "Two-Tower". This design would process the RNA sequence and the protein identity in parallel branches before fusing their representations to predict the binding affinity score.

For the RNA processing branch, we would adopt the hybrid *iDeepS* architecture as described in (Pan, 2018). This choice is motivated by the need to capture both local structural motifs and long-range sequential dependencies, which are critical for distinguishing the complex binding rules of differ-

ent proteins. The first branch, the RNA Encoder, takes the one-hot encoded RNA sequence (dimension 41×4) as input. The sequence is first passed through a 1D convolutional layer acting as a motif scanner. This layer utilizes multiple filters to detect specific local patterns (k-mers) along the RNA strand, regardless of their position. The resulting feature maps are then downsampled via a Max-pooling layer to retain only the strongest signals and reduce dimensionality. These pooled features are subsequently fed into a Bi-LSTM. The output of this branch is a dense vector summarizing the structural and sequential context of the RNA.

For the second branch, the Protein Encoder, since we have integer indexes for the proteins, we would employ a trainable embedding Layer. This layer functions as a lookup table that maps each unique integer protein ID to a learned continuous vector representation (embedding). During training, the model adjusts the values within this vector to represent the specific binding preferences of that protein. This allows the network to distinguish between proteins effectively, even without explicit biological feature inputs.

Finally, the information from both branches is integrated via a concatenation fusion strategy. The RNA vector from the *iDeepS* branch and the learned protein signature vector from the embedding branch are joined to form a single combined feature vector. This combined representation is passed through a series of fully connected (dense) layers, which model the non-linear interactions between the RNA features and the protein identity. The architecture ends in a single output node that predicts the continuous, Z-score normalized binding intensity, matching the distribution of the provided training labels.

The training objective should remain to minimize the provided `utils.masked_mse_loss`. This is due to the fact that alike to the single protein setting, the combined dataset will contain validities (masks). We must ensure that if a specific data point was a failed experiment (NaN), the model is not penalized for it, regardless of which protein it came from.

For the evaluation protocol, since we are using Protein IDs (embeddings), the model cannot predict for proteins it has never seen. Therefore, we cannot use a leave one out validation strategy. Instead we must use a stratified random split, that is, we take some percentage of the data from every protein for training, use another percentage from

every protein for a validation set for hyperparameter tuning, and hold out another percentage from every protein for independent testing.

2.3.2 Anticipated challenges and benefits

Training on multiple proteins simultaneously may act as a form of regularization. By sharing the weights of the RNA Encoder across all tasks, the model is forced to learn the RNA structural motifs rather than overfitting to the noise or distinctiveness of a single protein's dataset. This is particularly beneficial for proteins with smaller or noisier datasets, as they can leverage the high quality feature extractors learned from the proteins with more data, transferring learned structural knowledge to improve their own predictions.

A significant challenge in multi-task learning is the phenomenon of *negative transfer* (W. Zhang and Wu, 2023), where in our case improving performance on one protein could degrade performance on another. This would occur if the gradient updates for different proteins conflicted. For instance, if protein A requires the model to be highly sensitive to G-rich regions, while protein B requires the model to ignore them. If the shared RNA encoder cannot reconcile these conflicting requirements, the model may output lower accuracy for specific proteins compared to what training a dedicated model for each could provide.

3 Collaboration with other teams and use of AI tools

No collaborations were made with other teams, beyond sharing test set scores.

Google Gemini Pro proved helpful in finding articles that supported and expanded on some of our ideas for network architecture. These concerned mainly the fusing of the CNN and Bi-LSTM in 2.3, as well as the Maxpooling choice in 2.1, along other minor topics. Additionally, this LLM assisted in code and text revision.

4 Conclusions

In this work, we applied deep learning architectures to two distinct biological tasks: medical image classification and genomic sequence regression.

The addition of the softmax activation function on the last layer did not appear to meaningfully impact. Introducing Maxpooling on the CNN architectures seemed to improved performance.

In the RNA binding affinity prediction task, the Bi-LSTM significantly outperformed the 1D-CNN,

with Spearman correlations of 0.662 and 0.566, respectively. This performance gap confirms that capturing global sequential dependencies and implicit secondary structure is superior than relying solely on local motif detection for this problem. Adding an attention mechanism did not result in a meaningful improvement (0.666), due to the reduced size of the sequences.

Finally, we outlined a dual-branch strategy to extend this framework to multi-protein prediction, highlighting the challenges of negative sampling and data imbalance in scalable models.

4.1 References

References

- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Babak Alipanahi, Andrew Delong, Matthew T. Weirauch, and Brendan J. Frey. 2015. [Predicting the sequence specificities of dna- and rna-binding proteins by deep learning](#). *Nature Biotechnology*, 33(8):831–838.
- Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.
- Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.
- Rijnbeek P. Yan J. et al. Pan, X. 2018. [Prediction of rna-protein sequence and structure binding preferences using deep convolutional and recurrent neural networks](#). *BMC Genomics*, 19(511).
- Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. [Yara parser: A fast and accurate dependency parser](#). *Computing Research Repository*, arXiv:1503.06733. Version 2.
- L. Zhang W. Zhang, L. Deng and D. Wu. 2023. [A survey on negative transfer](#). *Journal of Automatica Sinica*, 10(2):305–329.
- Haoyang Zeng, Matthew D. Edwards, Ge Liu, and David K. Gifford. 2016. [Convolutional neural network architectures for predicting DNA–protein binding](#). *Bioinformatics*, 32(12):i121–i127.